

PegelConnect Pro - Gesamtdokumentation

FISI + FIAE Referenzprojekt

Stand: 12.08.2026
Projekt: PegelConnect Pro
Autor: Bünyamin Atik

Inhalt

- 1 Projektübersicht
- 2 Architektur
- 3 FIAE - Anwendung
- 4 FISl - Infrastruktur
- 5 Konfiguration
- 6 MQTT und Datenfluss
- 7 REST API
- 8 Vagrant, systemd und NGINX
- 9 Testbericht
- 10 Schreibtischtest
- 11 Troubleshooting
- 12 Erweiterungen
- 13 Betrieb und Abnahme

1. Projektübersicht

PegelConnect Pro ist ein eigenständiges Lern- und Referenzprojekt, das Softwareentwicklung und Systemintegration in einem durchgängigen System verbindet. Die Anwendung verarbeitet Wasserstands- und Wetterdaten, publiziert Zustände per MQTT und stellt Daten über REST sowie ein Web-Dashboard bereit.

- Wasserstände für KÖLN, BONN und MAINZ
- PEGELONLINE REST API
- Open-Meteo Wetterdaten
- MQTT via Mosquitto
- History 24h / 7d / 30d
- Karte, Verlauf, Reports, CSV/JSON/PDF Export
- Externe Laufzeitkonfiguration

Bereich	Technologie
Backend	Java 17 / Maven
Messaging	MQTT / Mosquitto
VM	Ubuntu 24.04 / Vagrant / VirtualBox
Service	systemd
Web	NGINX / HTML / CSS / JavaScript
Daten	PEGELONLINE / Open-Meteo

2. Architektur

Die Architektur trennt Datenquellen, Verarbeitung, Messaging, Web/API und Infrastruktur. NGINX dient als Einstiegspunkt für den Browser und leitet Requests an das Java Backend weiter.

```
PEGELONLINE ----> Java Backend ----> REST ----> NGINX ----> Browser
Open-Meteo ----/
config.json -----|-----> MQTT ----> Mosquitto
```

Dienst	Gast-Port	Host-Port
Java Backend	8080	8080
Mosquitto	1883	1885
NGINX	80	8088

3. FIAE - Anwendung

PegelConnectPro

Startpunkt; initialisiert Konfiguration, API-Clients, MQTT, Webserver und Scheduler.

AppConfig

Lädt externe Konfiguration und Stationsdaten.

PegelOnlineClient

Kapselt aktuelle und historische PEGELONLINE-Abfragen.

WeatherClient

Lädt Wetterdaten anhand der Stationskoordinaten.

StationReading

Internes Datenmodell für Pegelmessungen.

MqttGateway

Kapselt MQTT-Verbindung, Last Will, Publish und Reconnect.

WebServer

Stellt REST-Endpunkte und statische Ressourcen bereit.

Trendlogik

Vergleich	Trend
aktuell < vorher	-1
aktuell = vorher	0
aktuell > vorher	1
kein vorheriger Wert	null

4. FISI - Infrastruktur

Die VM wird reproduzierbar mit Vagrant bereitgestellt. Java Backend, Mosquitto und NGINX werden als Linux-Dienste betrieben und über systemd beim Boot automatisch gestartet.

Dienst	Aufgabe
pegelconnect-pro.service	Java Backend
mosquitto.service	MQTT Broker
nginx.service	Reverse Proxy / Webzugriff

Fehlertoleranz

Der Broker ist eine gewünschte, aber keine harte Startabhängigkeit. Dadurch bleibt das Backend bei einem Mosquitto-Ausfall aktiv und kann später erneut verbinden.

```
After=network-online.target mosquitto.service
Wants=network-online.target mosquitto.service
```

5. Konfiguration

Die Produktivkonfiguration liegt unter /etc/pegelconnect-pro/config.json. Lokal wird config/config.json verwendet. Ein expliziter Pfad kann über PEGELCONNECT_CONFIG gesetzt werden.

Station	Latitude	Longitude
KÖLN	50.9375	6.9603
BONN	50.7374	7.0982
MAINZ	49.9929	8.2473

6. MQTT und Datenfluss

```

pegel/status
pegel/koeln/wasserstand
pegel/bonn/wasserstand
pegel/mainz/wasserstand

```

```
{"unit":"cm","trend":0,"station":"KÖLN","value":55,"timestamp":"2026-08-12T15:00:00+02:00"}
```

T12 bestätigte den Last-Will-Ablauf online -> offline -> online. T14 bestätigte nach der systemd-Korrektur den Broker-Ausfall und die automatische Wiederverbindung.

7. REST API

Endpoint	Zweck
/api/state	Systemzustand, Stationen, History, MQTT Status
/api/config	Read-Only Runtime Config
/api/weather?station=...	Wetterdaten
/api/history?station=...&period=...	Historische Pegeldaten

Unterstützte History-Perioden: 24h, 7d, 30d. Ungültige Stationen oder Perioden liefern HTTP 400.

8. Vagrant, systemd und NGINX

Vagrant definiert VM, Ressourcen, Portweiterleitungen und Datei-Provisioning. Das Provisioning installiert Java, Maven, Mosquitto und NGINX, baut die JAR, installiert Konfiguration und systemd-Service und führt Funktionstests aus.

```

vagrant up
vagrant provision

```

NGINX leitet Port 80 der VM an 127.0.0.1:8080 weiter. Bei gestopptem Backend liefert NGINX erwartungsgemäß 502 Bad Gateway.

9. Testbericht

ID	Testfall	Ergebnis
T01	Webserver und Backend erreichbar	PASS
T02	MQTT Broker und Topics	PASS
T03	Weather API für drei Stationen	PASS
T04	History API 24h	PASS mit Hinweis
T05	History API 7d	PASS
T06	History API 30d	PASS
T07	Runtime Configuration	PASS
T08	Ungültige Station	PASS
T09	Ungültige History-Periode	PASS
T10	Autostart und Dienststatus	PASS
T11	VM Neustart / Wiederanlauf	PASS
T12	MQTT Last Will / Prozessabbruch	PASS
T13	NGINX bei Backend-Ausfall	PASS

ID	Testfall	Ergebnis
T14	MQTT Broker-Ausfall / Reconnect	PASS nach Korrektur

Detailbericht siehe separates PDF und docs/09_Testbericht.md.

10. Schreibtischtest

Der Schreibtischtest verfolgt den logischen Ablauf ohne einen neuen technischen Testlauf.

1. Konfiguration laden.
2. API- und MQTT-Komponenten initialisieren.
3. Webserver starten.
4. Stationen zyklisch abrufen.
5. Messwert validieren und Trend bestimmen.
6. Store aktualisieren und MQTT veröffentlichen.
7. REST und Dashboard stellen den Zustand bereit.

Fehlerpfad Broker

8. Mosquitto fällt aus.
9. Backend bleibt aktiv.
10. MQTT-Verbindung ist false.
11. Broker kehrt zurück.
12. Reconnect setzt MQTT-Verbindung wieder auf true.

11. Troubleshooting

Problem	Prüfung
NGINX 502	systemctl is-active pegelconnect-pro
Mosquitto startet nicht	grep nach mehrfachen listener 1883 Einträgen
Java Fehler	journalctl -u pegelconnect-pro
API prüfen	curl http://127.0.0.1:8088/api/state
MQTT prüfen	mosquitto_sub -t 'pegel/#' -v

Im Test wurde ein echter Mosquitto-Konflikt durch zwei Konfigurationsdateien mit listener 1883 gefunden und bereinigt.

12. Erweiterungen

- Wetterdaten
- Leaflet / OpenStreetMap Karte
- Stadtwappen
- History 24h / 7d / 30d
- statistische Reports
- CSV / JSON / PDF Export
- Event Log
- externe Konfiguration
- NGINX Reverse Proxy

Empfohlene nächste Verbesserungen

- Deduplication nach station + timestamp
- persistente serverseitige Logs
- automatisierte Testskripte
- JUnit Unit Tests

- GitHub Actions CI

13. Betrieb und Abnahme

```
vagrant up  
vagrant provision
```

Dashboard: <http://127.0.0.1:8088>

- VM startet
- alle drei Dienste active
- Dashboard erreichbar
- KÖLN/BONN/MAINZ vorhanden
- MQTT verbunden
- Weather und History funktionieren
- HTTP-400-Fehlerfälle funktionieren
- Recovery für Java und MQTT funktioniert

Quellen

- PEGELONLINE
- Open-Meteo
- OpenStreetMap
- Wikimedia Commons